

Scan Vision's Core

MIRAGE 2.0 — Reverse Engineering — Walkthrough

CATEGORY	FILES	DEPLOYMENT
Reverse Engineering	vision_core (stripped)	Offline binary — run locally
VULNERABILITY	STATUS	
N/A — embedded-constant XOR comparison	Solved	

Step 1 — First Contact

“Vision's core has gone dark... Recover the diagnostic password and restore the core.” Running the binary prints a banner and a single prompt: “Diagnostic password: ”. Typing anything and pressing enter gets an immediate “authentication rejected” — no hints, no partial credit, just accept or reject. Unlike the Mind Stone or Shuri challenges, there's no flavor text here about self-modification or watchers, which is itself a useful signal: this is likely the simplest reversing exercise in the set, without anti-debugging to work around.

Step 2 — Recon

```
$ file vision_core
vision_core: ELF 64-bit LSB executable, x86-64, ... stripped
```

Stripped means there are no function names to read directly — `objdump -d` will show addresses like `sub_401176` instead of meaningful names. The practical approach is to find `main` first (the entry point's disassembly leads to it via the usual `__libc_start_main` call), then follow which function it calls immediately after printing the password prompt.

Step 3 — Reading the Validation Function

That function does exactly two things, in order: it calls `strlen()` on the input and rejects immediately unless the length is exactly 10; then it loops over the 10 bytes, XORing each one with the constant `0x27` and comparing the result against a fixed 10-byte table sitting in `.rodata`: `71 16 74 6e 68 69 04 13 15 06`. Any mismatch fails on the spot.

Because XOR is its own inverse, the check can be run backward exactly as easily as forward: XORing that same table with the same constant `0x27` reconstructs the only 10-byte string that will ever pass — no guessing, no brute force, just running the comparison in reverse.

```
>>> table = bytes.fromhex('71 16 74 6e 68 69 04 13 15 06'.replace(' ', ''))
>>> bytes(b ^ 0x27 for b in table)
b'V1SION#42!'
```

Step 4 — Running It

```
$ ./vision_core  
Diagnostic password: V1SI0N#42!
```

FLAG

```
MIRAGE{V1S10N_C0R3!_R3ST0R3D#42}
```

NOTE: As a sanity check, the flag string also happens to sit in plaintext at a fixed `.rodata` offset in this binary, independent of the password logic — confirming the recovered password is correct without needing to run the binary at all. That won't be true of every challenge (several others in this set deliberately keep the flag encoded so it can't be found this way), so treat it as a bonus check here, not a general technique.

Takeaway

A stripped binary with no anti-debug measures and a simple accept/reject prompt is usually hiding nothing more than a fixed comparison against an obfuscated constant. Before reaching for brute force, check whether the obfuscation is a simple reversible operation like XOR, addition, or a bit rotation — those can be inverted directly from the stored constant with no guessing at all.